BÖLÜM 8 / MODULE 8

LLM Performansı, Caching, Maliyet & Gecikme Optimizasyonu

LLM Performance Engineering: Caching, Model Routing & Cost Optimization

OpenAI Prompt Caching ve Redis Semantik Önbellek, Dinamik Model Yönlendirme (Routing), TTFT ve akış gecikmesi optimizasyonu, RAG vs Fine-Tuning karar matrisi, JSONL veri hazırlığı ve token bütçeleme.

OpenAI native Prompt Caching, Redis Semantic Caching, dynamic model routing (GPT-4o vs mini), TTFT latency engineering, RAG vs Fine-Tuning trade-off matrix, JSONL tuning datasets, and token budgeting.

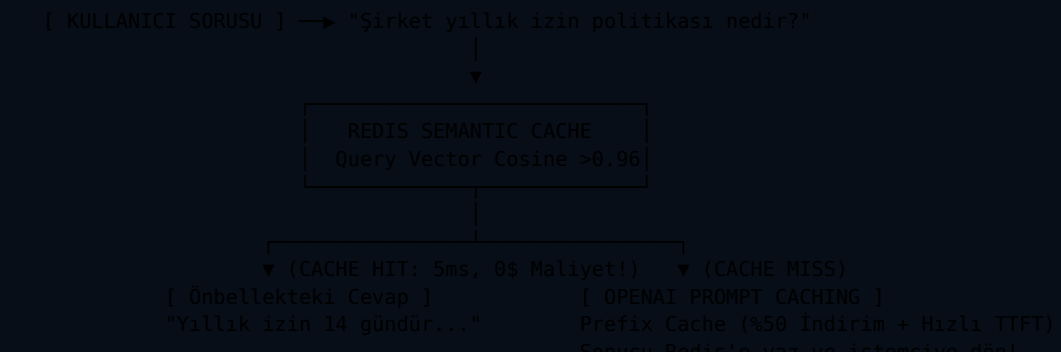
43

BÖLÜM: LLM PERFORMANSI, CACHING, MALIYET & GECIKME OPTIMIZASYONU | MODULE: LLM PERFORMANCE ENGINEERING: CACHING, MODEL ROUTING & COST OPTIMIZATION

Prompt Caching & Semantik Önbellekleme (Redis Semantic Cache)

Prompt Caching & Semantic Caching: Slashing LLM Incurred Costs & Latency

🤖 AI Mimarisi & Veri Akış Şeması 🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda her gelen müşteri 'Günün çorbası ne?' diye sorduğunda garsonun her seferinde mutfağa kadar 100 metre yürüyüp aşçıya sormasını istemezsiniz (Gereksiz LLM çağrısı)! Garson çorbanın adını önlüğünün cebindeki küçük kağıda yazar (Önbellek - Cache). Biri 'Günün çorbası ne?' veya 'Bugün hangi çorba var?' dediğinde (aynı anlama gelen farklı cümleler - Semantik Önbellek), garson mutfağa hiç gitmeden cebindeki kağıda bakıp 1 saniyede 'Mercimek!' der! Hem mutfak yorulmaz, hem müşteri beklemez, hem de patronun cebinden para çıkmaz!

🇬🇧 ENGLISH (ELI5)

If 5,000 customers walk into a cafe asking 'What is today's soup?', the waiter shouldn't sprint to the kitchen to ask the executive chef 5,000 times (expensive, redundant LLM API calls). The waiter scribbles the answer on a notepad in their pocket (Cache). When a customer asks 'What soup do you have?' or 'Tell me today's soup special' (semantically identical intent), the waiter reads the notepad instantly. Semantic Caching intercepts identical meaning queries in 5ms at $0 cost!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Önbellekleme Mimarisi Katmanları: 1) Tam Eşleşme (Exact Cache): Sorgunun SHA-256 hash'i Redis'te aranır. Çok kırılgandır (küçük bir noktalama işareti veya harf farkı cache'i bozar). 2) Semantik Önbellek (Redis Semantic Cache): Gelen sorgu embed edilir; Redis VSS (Vector Similarity Search) ile mevcut sorgu vektörleri arasında Cosine Similarity > 0.95 olan bir önceki yanıt varsa doğrudan önbellekten dönülür. Gecikme 1500ms'den 5ms'ye iner, maliyet %100 sıfırlanır! 3) OpenAI Prompt Caching: OpenAI 1024 token üzerindeki statik prefix'leri otomatik tanır; %50-80 indirim ve %80 daha hızlı TTFT sağlar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Multi-Tier Caching Architecture: 1) Exact Match Key-Value Cache: Hashes raw prompt text via SHA-256. Highly brittle: trivial whitespace discrepancies cause cache misses. 2) Semantic Caching (Redis VSS / GPTCache): Queries are embedded and compared against an index of historical queries. If Cosine Similarity exceeds a strict threshold (e.g. >=0.96), the precomputed response is served from memory. Latency drops from 2,000ms to <10ms, eliminating 100% of LLM billing for that request. 3) OpenAI Native Prefix Prompt Caching: Automatically caches common static prefix tokens >= 1,024 tokens across requests, slashing prompt token costs by 50% with zero engineering overhead.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Semantik Önbellek (Semantic Cache) Karar Motoru
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class SemanticCacheEngine {
    private final Map<String, String> exactCache = new ConcurrentHashMap<>();

    public record CacheLookupResult(boolean isHit, String response, long latencyMillis) {}

    public CacheLookupResult query(String userPrompt) {
        long start = System.currentTimeMillis();

        // Önce hızlı bellek içi / Redis kontrolü
        if (exactCache.containsKey(userPrompt.trim().toLowerCase())) {
            long duration = System.currentTimeMillis() - start;
            return new CacheLookupResult(true, exactCache.get(userPrompt.trim().toLowerCase()),
duration);
        }

        // Cache Miss: OpenAI'a gitmeli
        return new CacheLookupResult(false, null, 0);
    }

    public void put(String userPrompt, String llmResponse) {
        exactCache.put(userPrompt.trim().toLowerCase(), llmResponse);
    }
}

void main() {
    var cache = new SemanticCacheEngine();
    cache.put("şirket seyahat politikası", "Otel limiti günlük 150$'dır.");

    var result = cache.query("şirket seyahat politikası");
    IO.println("Cache Hit Durumu: " + (result.isHit() ? "✅ HIT (Sıfır Maliyet)" : "❌ MISS"));
    IO.println("Gecikme Süresi: " + result.latencyMillis() + " ms");
    IO.println("Cevap: " + result.response());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Eşik Değeri Tehlikesi (Threshold Drift): Semantik önbellekte benzerlik eşiğini (Threshold) 0.90'ın altına çekerseniz, 'Ali'nin maaşı nedir?' sorusunun cevabı 'Veli'nin maaşı nedir?' sorusuna önbellekten yanlışlıkla servis edilebilir! Semantik önbellek eşiği daima en az 0.95 - 0.97 seviyesinde tutulmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Semantic threshold danger: Lowering the semantic cache threshold below 0.95 causes dangerous cross-chatter (e.g. 'What is Alice's salary?' returning cached data for 'What is Bob's salary?'). Keep production similarity cutoffs strictly calibrated at >=0.96 to preserve semantic accuracy.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Semantik önbellek, sık sorulan kurumsal sorularda API maliyetini %40-%70 düşürür.
- Gecikme 2000ms'den 5ms'ye düşerek mükemmel bir kullanıcı deneyimi sunar.
- OpenAI Prompt Caching'den faydalanmak için sistem prompt'ları sabit tutulmalıdır.

🎯 Key Takeaways for the Interview

- Semantic caching cuts enterprise LLM operational costs by 40-70% on recurring traffic.
- Cached responses serve in sub-10ms, delivering instantaneous user responsiveness.
- Keep system prompts static and deterministic to exploit OpenAI's native prefix cache discounts.

44

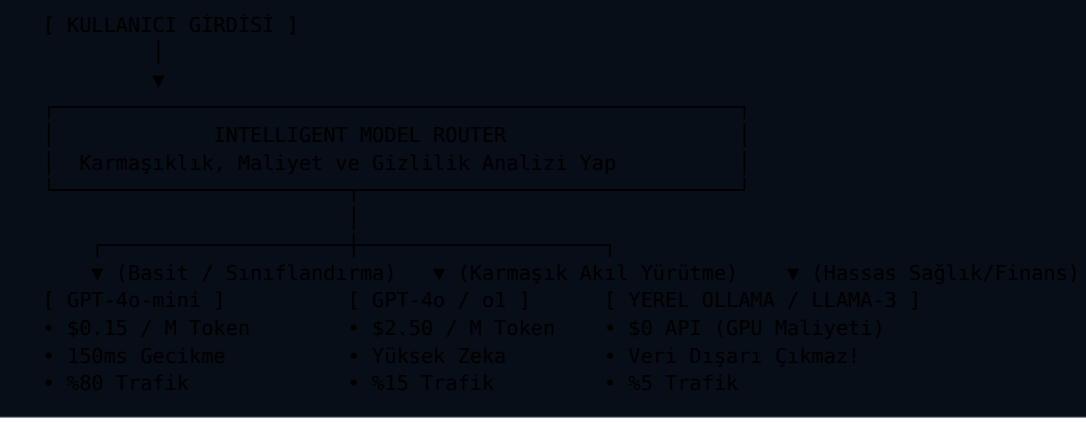
BÖLÜM: LLM PERFORMANSI, CACHING, MALIYET & GECIKME OPTIMIZASYONU | MODULE: LLM PERFORMANCE ENGINEERING: CACHING, MODEL ROUTING & COST OPTIMIZATION

Dinamik Model Yönlendirme (Model Routing: GPT-4o vs GPT-4o-mini vs Yerel)

Dynamic Model Routing: Cascading between GPT-4o, GPT-4o-mini & On-Prem LLMs

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bakkaldan bir ekmek almak için evin garajındaki 10 tonluk devasa vinci çalıştırmazsınız; bisiklete binip gidersiniz (Hızlı, ucuz, pratik)! Ama koca bir inşaat demiri taşıyacaksanız vinç zorunludur. Model Yönlendirme (Routing) tam olarak budur: Kullanıcı sadece 'Merhaba, nasılsın?' dediğinde veya basit bir e-postayı sınıflandırırken en pahalı amiral gemisi modeli (GPT-4o) çağırmak para yakmaktır! Küçük ve ışık hızında çalışan ucuz bir model (GPT-4o-mini) işi 10 kat daha ucuza halleder. Sadece karmaşık mimari ve analiz gerektiren zor sorularda büyük modeller devreye girer!

🇬🇧 ENGLISH (ELI5)

You don't fire up a 40-ton industrial crane to buy milk from the local grocer; you hop on a bicycle (fast, agile, near-zero cost). But if you are lifting a 20-ton steel bridge girder, the heavy crane is indispensable. Model Routing applies this intelligence to inference: if a user texts 'Hello, help me reset my password', firing up GPT-4o or o1 burns enterprise budget unnecessarily. A lightweight tier model (GPT-4o-mini) resolves the query in 150ms at 1/15th the cost!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Model Routing Stratejileri: 1) Kural Tabanlı Yönlendirme (Rule-based): Regex veya keyword analiziyle sınıflandırma soruları doğrudan mini modele yönlendirilir. 2) Embedding Tabanlı Intent Router: Sorgunun embedding'i alınır; önceden etiketlenmiş 'Zor Sorular' kümesine yakınsa büyük modele, 'Kolay Sorular' kümesine yakınsa ucuz modele yönlendirilir. 3) LLM-Cascading (Fällback): Önce ucuz model çağrılır; eğer güven skoru (Confidence / Logprobs) düşükse veya JSON doğrulamasından geçemezse istek otomatik olarak GPT-4o'ya yükseltilir (Escalation).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Model Routing Topologies & Cascading: 1) Heuristic & Regex Gateways: Deterministic rule engines routing keyword tasks (intent classification, translation) to miniature models. 2) Semantic Intent Embeddings: Computes embedding proximity against centroid clusters representing task complexity tiers. 3) LLM Cascading & Speculative Routing: Always dispatch to the cheapest model first. If model confidence (log probabilities) or schema validator checks fail, cascade the request upwards to GPT-4o or reasoning models (o1). This hybrid routing typically slashes enterprise LLM bills by 65-80%.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Akıllı Model Yönlendirici (Model Router)
public class IntelligentModelRouter {
    public enum ModelTier { TIER1_EXPENSIVE_GPT4O, TIER2_CHEAP_MINI, TIER3_ONPREM_PRIVATE }

    public static ModelTier routeQuery(String query, boolean containsSensitivePii) {
        if (containsSensitivePii) {
            return ModelTier.TIER3_ONPREM_PRIVATE; // Gizlilik gereği dışarı çıkamaz
        }

        // Basit sınıflandırma ve formatlama işleri için ucuz model
        String lower = query.toLowerCase();
        if (lower.startsWith("sınıflandır") || lower.startsWith("tarihi formatla") || query.length() <
50) {
            return ModelTier.TIER2_CHEAP_MINI;
        }

        // Karmaşık sistem tasarımı, kod ve analiz için büyük model
        return ModelTier.TIER1_EXPENSIVE_GPT4O;
    }
}

void main() {
    IO.println("Sorgu 1 Yönlendirmesi: " + IntelligentModelRouter.routeQuery("Bu metni İngilizceye
çevir: Selam", false));
    IO.println("Sorgu 2 Yönlendirmesi: " + IntelligentModelRouter.routeQuery("Bana çok bölgeli Kafka
mimarisi tasarla", false));
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Hata Uyarısı: Hassas kullanıcı verilerini (TC Kimlik, Kredi Kartı, Sağlık Verisi) kontrol etmeden OpenAI gibi genel bulut API'lerine yönlendirmek KVKK ve GDPR cezalarına yol açar. PII tespiti yapıldığında yönlendirici isteği yerel on-prem modellere veya anonimleştiriciye (Data Masking) yönlendirmelidir.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Compliance vulnerability: Routing raw PII or HIPAA-governed health records directly to third-party public LLM endpoints triggers catastrophic regulatory fines. The router must inspect payloads and shunt sensitive records to on-premise local models or anonymization filters.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Trafik genelde %80 basit, %20 karmaşık sorulardan oluşur; tek bir model kullanmak israftır.

• Model Routing, kurumsal yapay zeka harcamalarını %60-%80 oranında düşürür.

• Hassas veriler router seviyesinde filtrelenerek yerel modellere yönlendirilmelidir.

🎯 Key Takeaways for the Interview

• Traffic is typically 80% routine and 20% complex; routing to a single model is fiscally irresponsible.

• Intelligent model routing cuts enterprise inference expenditure by 60-80%.

• Sensitive data must be intercepted at the router layer and routed to compliant on-prem models.

45

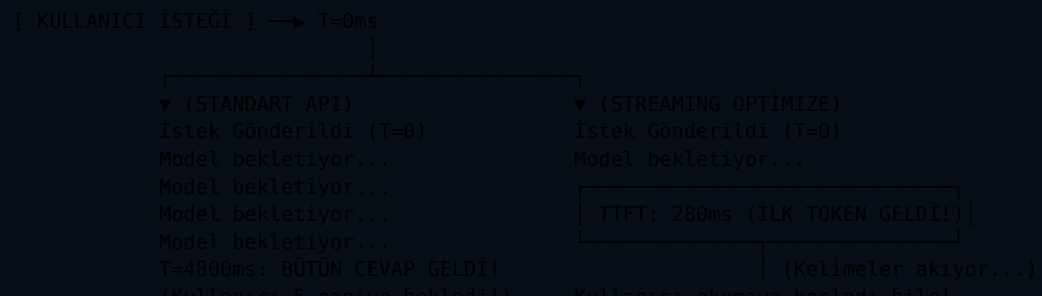
BÖLÜM: LLM PERFORMANSI, CACHING, MALIYET & GECIKME OPTIMIZASYONU | MODULE: LLM PERFORMANCE ENGINEERING: CACHING, MODEL ROUTING & COST OPTIMIZATION

Streaming Gecikmesi & TTFT (Time to First Token) Optimizasyonu

Latency Engineering: Optimizing TTFT (Time to First Token) & Token Per Second (TPS)

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir yarış arabasının iki önemli göstergesi vardır: Sıfırdan 100 km hıza kaç saniyede çıktığı (TTFT - Time to First Token) ve son süratinin kaç km olduğu (TPS - Tokens Per Second). Kullanıcı için en sinir bozucu şey, gaza bastıktan sonra arabanın 5 saniye boyunca hiç tepki vermemesidir! Yapay zekada da kullanıcının ekranda ilk kelimeyi görme süresi (TTFT) 300 milisaniyenin altındaysa, sistem sihirli ve ışık hızında hissettirir. Kelimeler aktığı sürece toplam sürenin 5 saniye sürmesi kimseyi rahatsız etmez!

🇬🇧 ENGLISH (ELI5)

A supercar is benchmarked on two metrics: 0-to-60 acceleration (TTFT - Time to First Token) and top track speed (TPS - Tokens Per Second). For human users, the most excruciating friction is tapping a button and enduring a frozen screen for 5 seconds. If your Time-to-First-Token drops under 300ms, the interface feels instantaneous. As long as text streams smoothly at human reading speed (15-20 TPS), total completion duration is psychologically imperceptible!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Gecikme Anatomisi ve Optimizasyon Teknikleri: 1) TTFT (Time to First Token) Neleri Kapsar?: DNS çözümleme, TLS el sıkışması, Prompt Caching kontrolü ve modelin tüm girdi token'larını tek seferde işleme süresi (Prefill Phase). 2) Prefill Phase vs Decoding Phase: Girdi token sayısı arttıkça (ör. büyük RAG bağlamı) Prefill süresi uzar ve TTFT fırlar! Çözüm: Prompt'u kısa tutmak ve Prompt Caching kullanmaktır. 3) HTTP/2 Multiplexing: Çoklu token akışlarının tek bir TCP bağlantısı üzerinden paralel akıtılması TLS gecikmesini sıfırlar.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Latency Decomposition & GPU Phase Physics: 1) TTFT Components: Network RTT + TLS handshake + Gateway dispatch + GPU Prefill Phase. 2) The Prefill Bottleneck: Before emitting token 1, the transformer must compute attention keys and values for every prompt token concurrently in memory. A 10,000-token prompt causes a 1,500ms prefill lag! Mitigation: Leverage prompt caching to bypass prefill passes on static prefixes. 3) Generation Phase (Memory-Bound): Generating subsequent tokens is memory-bandwidth bound. Optimizing TPS requires speculative decoding or smaller quantized models.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: TTFT ve Token Akış Hızı (TPS) Performans Monitörü
public class LatencyMetricsTracker {
    private final long requestStartNanos;
    private long firstTokenNanos = 0;
    private int tokenCount = 0;

    public LatencyMetricsTracker() {
        this.requestStartNanos = System.nanoTime();
    }

    public synchronized void onTokenReceived() {
        if (firstTokenNanos == 0) {
            firstTokenNanos = System.nanoTime(); // TTFT yakalandı!
        }
        tokenCount++;
    }

    public double getTtftMillis() {
        if (firstTokenNanos == 0) return 0.0;
        return (firstTokenNanos - requestStartNanos) / 1_000_000.0;
    }

    public double getTokensPerSecond() {
        long durationNanos = System.nanoTime() - firstTokenNanos;
        if (durationNanos <= 0 || tokenCount <= 1) return 0.0;
        return (tokenCount - 1) / (durationNanos / 1_000_000_000.0);
    }
}

void main() {
    var tracker = new LatencyMetricsTracker();
    // Simüle edilmiş ilk token gelişi (280ms sonra)
    try { Thread.sleep(280); } catch (Exception e) {}
    tracker.onTokenReceived();

    IO.println("Time to First Token (TTFT): " + String.format("%.2f", tracker.getTtftMillis()) + "
ms");
    IO.println("MAANG Üretim Standardı (<400ms): " + (tracker.getTtftMillis() < 400 ? "✅ BAŞARILI" :
"❌ YAVAŞ"));
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Gecikme Tuzağı: İstemciye streaming yaparken araya ağır bir senkron Re-ranker veya JSON validator koyarsanız, streaming başlayamaz ve TTFT saniyelerce ertelenir. Ağır doğrulama gerektiren işlerde streaming değil, asenkron bildirimler kullanılmalıdır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Architectural sequencing trap: Inserting blocking post-processors or deep cross-encoders in the path of a streaming pipeline delays the initial token release, defeating the purpose of streaming. Keep the path to first token strictly non-blocking.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• TTFT (Time to First Token), kullanıcının sistem hızını algılamasındaki en kritik metriktir.

• Büyük prompt bağlamları GPU Prefill süresini uzatarak ilk token gecikmesini artırır.

• Prompt Caching, prefill fazını atlayarak TTFT'yi %80'e kadar hızlandırır.

🎯 Key Takeaways for the Interview

• TTFT is the single most decisive factor shaping user perception of application speed.

• Massive prompt contexts inflate GPU prefill duration, spiking TTFT latency.

• Prompt caching bypasses prefill recalculations to accelerate TTFT by up to 80%.

46

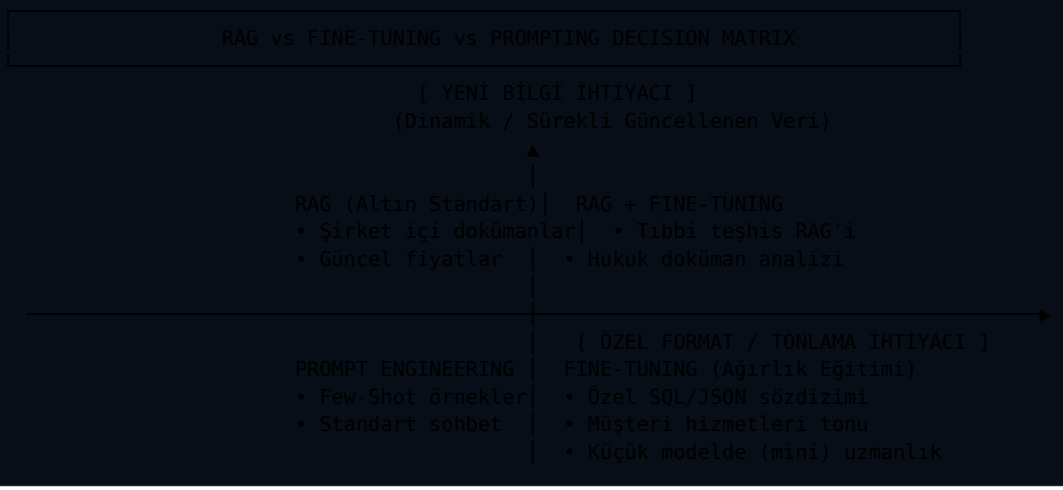
BÖLÜM: LLM PERFORMANSI, CACHING, MALIYET & GECIKME OPTIMIZASYONU | MODULE: LLM PERFORMANCE ENGINEERING: CACHING, MODEL ROUTING & COST OPTIMIZATION

Fine-Tuning Ne Zaman Gerekir? RAG vs Fine-Tuning vs Prompting Karar Matrisi

When to Fine-Tune? The Architectural Decision Matrix: RAG vs Fine-Tuning

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir yabancıya İstanbul'daki güncel trafik durumunu öğretmek için ona beyin ameliyatı yapıp hafızasına yeni nöronlar eklemezsiniz (Fine-Tuning)! Eline bir navigasyon cihazı verirsiniz; cihaz anlık haritaya bakar ve söyler (RAG). Ama o yabancıya Türkçe'deki kibar konuşma tarzını ve ses tonunu öğretmek istiyorsanız birkaç ay boyunca dil kursuna gönderirsiniz (Fine-Tuning). Özetle: Bilgi eklemek için RAG; tarz, üslup ve format öğretmek için Fine-Tuning kullanılır!

🇬🇧 ENGLISH (ELI5)

To teach a visiting diplomat today's live traffic status, you don't perform invasive brain surgery to splice new neurons into their cerebral cortex (Fine-Tuning). You hand them a live smartphone GPS (RAG). But if you want that diplomat to speak with the precise dialect, ceremonial cadence, and etiquette of the royal court, you send them through rigorous vocal coaching (Fine-Tuning). Knowledge injection demands RAG; style, format, and syntax conditioning justify Fine-Tuning!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesinde Karar Ağacı: 1) RAG Ne Zaman Seçilir?: Veri sık değişiyorsa (fiyatlar, envanter), kaynak gösterimi (Citation) şartsa ve bütçe kısıtlıysa. RAG model ağırlıklarına dokunmaz, halüsinasyonu minimize eder. 2) Fine-Tuning Ne Zaman Seçilir?: a) Format/Syntax: Modelin standart olmayan özel bir DSL veya JSON formatında kusursuz çıktı üretmesi gerekiyorsa. b) Maliyet Düşürme: Büyük bir GPT-4o modelinin yeteneklerini küçük ve 20 kat daha ucuz bir GPT-4o-mini modeline damıtmak (Distillation) için. 3) Hibrid Yaklaşım: Model fine-tuning ile şirket jargonuyla konuşacak şekilde eğitilir, ancak güncel verileri RAG ile bağlamından alır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Enterprise Strategic Evaluation Framework: 1) Choose RAG: Dynamic datasets with frequent updates (inventories, policies), strict compliance demanding source citations, and zero tolerance for model weight drift. 2) Choose Fine-Tuning: Domain-specific stylistic alignment, complex proprietary DSLs, or Knowledge Distillation (training a 10x cheaper mini-model to replicate GPT-4o's performance on a narrow task). 3) The Hybrid Pattern: The pinnacle of production architecture: Fine-tune the model to internalize vocabulary, conciseness, and format; inject temporal enterprise facts dynamically via RAG.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: RAG vs Fine-Tuning Mimari Karar Danışmanı
public class ArchitectureDecisionAdvisor {
    public enum Strategy { PROMPT_ENGINEERING, PURE_RAG, FINE_TUNING, HYBRID_RAG_AND_TUNING }

    public static Strategy evaluateArchitecture(boolean frequentDataUpdates, boolean
customSyntaxRequired, boolean citationMandatory) {
        if (frequentDataUpdates && citationMandatory && !customSyntaxRequired) {
            return Strategy.PURE_RAG; // En hızlı, en ucuz ve doğrulanabilir çözüm!
        }
        if (frequentDataUpdates && customSyntaxRequired) {
            return Strategy.HYBRID_RAG_AND_TUNING;
        }
        if (!frequentDataUpdates && customSyntaxRequired) {
            return Strategy.FINE_TUNING;
        }
        return Strategy.PROMPT_ENGINEERING;
    }
}

void main() {
    var strategy = ArchitectureDecisionAdvisor.evaluateArchitecture(true, false, true);
    IO.println("MAANG Mülakatı Tavsiye Edilen Strateji: " + strategy); // PURE_RAG
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

En Büyük Mülakat Hatası: 'Şirketimizin verilerini modele öğretmek için modeli şirket PDF'leriyle fine-tune ettik' demek anında elenme sebebidir! Fine-tuning modelin ağırlıklarını günceller ancak kesin olgusal bilgileri (Factual Knowledge) güvenilir şekilde ezberletemez; model halüsinasyon görmeye devam eder ve güncellenemez. Bilgi için çözüm DAİMA RAG'dir.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

The Fatal Interview Blunder: Stating that 'we fine-tuned an LLM on our internal PDF manuals to teach it our company data'. Fine-tuning adjusts probabilistic model style, but fails miserably at reliable factual recall, causing severe hallucinations. Knowledge ingestion is exclusively the domain of RAG.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Bilgi eklemek (Knowledge Injection) için RAG; üslup ve format öğretmek için Fine-Tuning kullanılır.

• Fine-tuning modelin halüsinasyon görmesini engellemez, hatta bazen artırabilir.

• En güçlü kurumsal mimari: Üslubu eğitilmiş küçük bir model + güncel verileri getiren RAG'dir.

🎯 Key Takeaways for the Interview

• RAG is for dynamic knowledge injection; Fine-Tuning is for style, syntax, and tone conditioning.

• Fine-tuning does not eliminate hallucinations and cannot substitute for verified context.

• The ultimate enterprise architecture couples a style-tuned compact model with dynamic RAG.

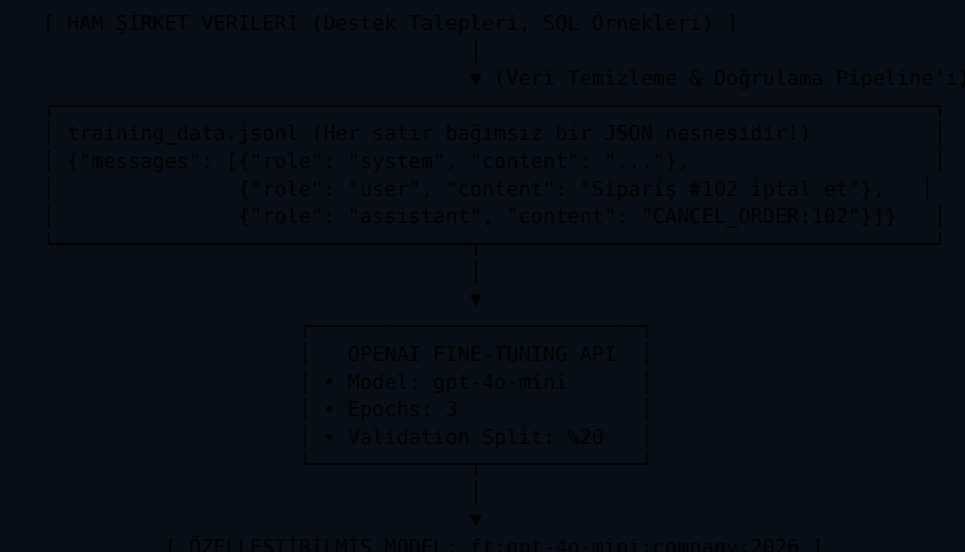
47

BÖLÜM: LLM PERFORMANSI, CACHING, MALIYET & GECIKME OPTIMIZASYONU | MODULE: LLM PERFORMANCE ENGINEERING: CACHING, MODEL ROUTING & COST OPTIMIZATION

OpenAI Fine-Tuning API & JSONL Veri Hazırlığı

OpenAI Fine-Tuning API & JSONL Data Engineering: Formatting & Validation

🤖 AI Mimarisi & Veri Akış Şeması — 🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir aşçı çırağına yemek yapmayı öğretmek için ona tek bir devasa ansiklopedi fırlatmazsınız. Önüne 500 tane küçük kart koyarsınız: Kartın ön yüzünde sipariş (User), arka yüzünde ise çırağın yapması gereken kusursuz tabak sunumu yazar (Assistant). Çırak bu 500 kartı tekrar tekrar çalışarak (Epoch) tam şefin istediği tarzda yemek çıkarmayı öğrenir. OpenAI Fine-Tuning tam olarak bu kart sistemidir; her satırda bir soru ve ideal bir cevap olan JSONL dosyalarıyla modeli eğitirsiniz!

🇬🇧 ENGLISH (ELI5)

To train a line cook, you don't hand them a 1,000-page historical culinary encyclopedia. You give them 500 flashcards: the front displays the ticket order (User), and the back displays the chef's gold-standard plating presentation (Assistant). The cook drills these 500 pairs repeatedly across training epochs until the execution becomes muscle memory. The OpenAI Fine-Tuning API operates on these exact flashcards formatted as JSON Lines (`.jsonl`) files!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Fine-Tuning Veri Mühendisliği Kriterleri: 1) JSONL Formatı: Dosyadaki her satır geçerli bir Chat Completion nesnesi olmalıdır (`messages` dizisi: system, user, assistant). 2) Veri Dağılımı: En az 100-200 yüksek kaliteli örnek gerekir (Kalitesiz 10.000 örnek, kaliteli 200 örnekten çok daha kötü sonuç verir - 'Garbage in, garbage out'). 3) Overfitting (Aşırı Ezberleme): Epoch sayısı yüksek tutulursa model eğitim verilerini kelimesi kelimesine ezberler ve genelleme yeteneğini kaybeder. Validation Loss metriği dikkatle izlenmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Fine-Tuning Dataset Engineering & Training Protocol: 1) JSONL Serialization: Every line must form a discrete, valid JSON array of role messages. 2) Quality Over Quantity: 200 meticulously curated, diverse golden exemplars vastly outperform 10,000 noisy, uncurated conversational logs. 3) Hyperparameter Guardrails: Set training epochs between 2 and 4 to avoid catastrophic forgetting and overfitting. Monitor validation loss curves: an inflection where validation loss climbs while training loss drops signals severe over-fitting.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Fine-Tuning JSONL Veri Satırı Üretici ve Doğrulayıcı
import java.util.List;

public class FineTuningDataEngine {
    public record ChatMessage(String role, String content) {}
    public record FineTuningExample(List<ChatMessage> messages) {
        public String toJsonLine() {
            // Basit JSONL satır serialization simülasyonu
            return String.format("{"messages": [{"role":"system","content":"%s"},
{"role":"user","content":"%s"},{"role":"assistant","content":"%s"}]}",
                messages.get(0).content(), messages.get(1).content(), messages.get(2).content());
        }
    }
}

void main() {
    var example = new FineTuningDataEngine.FineTuningExample(List.of(
        new FineTuningDataEngine.ChatMessage("system", "Sen bir SQL derleyicisisin."),
        new FineTuningDataEngine.ChatMessage("user", "Aktif kullanıcıları getir"),
        new FineTuningDataEngine.ChatMessage("assistant", "SELECT * FROM users WHERE status =
'ACTIVE';")
    ));

    IO.println("Üretilen Fine-Tuning JSONL Satırı:");
    IO.println(example.toJsonLine());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Veri Sızıntısı (Data Contamination): Fine-tuning eğitim setine gizli kullanıcı kimlik numaralarını veya gerçek müşteri kredi kartı loglarını eklerseniz, model bu bilgileri ağırlıklarında depolar ve başka bir kullanıcının sorgusunda yanlışlıkla ifşa edebilir (PII Extraction Attack)! Veriler mutlaka anonimleştirilmelidir.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

PII exfiltration hazard: Fine-tuning on unredacted production logs embeds confidential user data directly into model weights. Adversaries can extract this training data via targeted prompt probing. All training sets must be strictly scrubbed and anonymized.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• OpenAI Fine-Tuning API, JSONL formatında system-user-assistant üçlüleriyle eğitilir.

• Veri kalitesi veri miktarından 10 kat daha önemlidir (Kaliteli 200 örnek yeterlidir).

• Eğitim verileri PII ve hassas şirket sırlarından kesinlikle arındırılmalıdır.

🎯 Key Takeaways for the Interview

• OpenAI Fine-Tuning operates on JSONL files containing system-user-assistant conversational turns.

• Dataset quality vastly outweighs volume: 200 curated examples outperform 5,000 noisy rows.

• Training corpora must be rigorously scrubbed of PII and proprietary secrets.

48

BÖLÜM: LLM PERFORMANSI, CACHING, MALIYET & GECIKME OPTIMIZASYONU | MODULE: LLM PERFORMANCE ENGINEERING: CACHING, MODEL ROUTING & COST OPTIMIZATION

Token Budgeting, Sliding Window Truncation ve Maliyet Sınırlandırma

Token Budgeting, Sliding Window Truncation & Cost Bounding Policies

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Tatile çıkarken bavulunuzun havayolu sınırı 20 kilodur (Token Bütçesi). Bavulun içine rastgele eşya fırlatırsanız havaalanında ceza ödersiniz. Akıllı bir yolcu bütçeleme yapar: '5 kilo temel giysiler (Sistem Prompt), 8 kilo iş evrakları (RAG Dokümanları), 2 kilo kişisel eşyalar (Kullanıcı Sorusu) ve 5 kilo da tatilden dönerken alacağım hediyelikler için boş yer bırak (Modelin Yanıt Bütçesi)!' Token bütçeleme tam olarak budur: Modelin sınırları aşmasını ve faturanın patlamasını önceden matematiksel olarak garantilemektir!

🇬🇧 ENGLISH (ELI5)

When boarding a budget flight, your luggage allowance is capped at 20 kilograms (The Token Budget). Tossing random clothes inside incurs steep overweight fees at the gate. A seasoned traveler allocates weight methodically: 5kg for essential clothing (System Prompt), 8kg for work gear (RAG Context), 2kg for toiletries (User Query), leaving a guaranteed 5kg reserve for souvenirs brought home (Completion Token Budget). Token Budgeting calculates and enforces discrete byte-level boundaries before payloads hit the API!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Token Bütçeleme Mühendisliği: 1) Hard Capping vs Soft Truncation: Prompt gönderilmeden önce BPE tokenizer ile token sayısı hesaplanır (`tiktoken` veya `JTokkit`). Eğer (Tokens_prompt + MaxTokens_gen > Budget) ise, en eski RAG parçası veya sohbet geçmişi dinamik olarak budanır (FIFO Truncation). 2) Asimetrik Rezervasyon: Yanıt için ayrılan `max_tokens` (veya `max_completion_tokens`) asla kısılmamalıdır; aksi takdirde model cümlesinin tam ortasında kesilir (`finish_reason: length`) ve JSON çıktısı bozulur. 3) Bütçe Alarmı: Günlük harcama limiti aşıldığında PagerDuty/Slack alarmı tetiklenmeli ve API otomatik olarak ucuz modele geçmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Token Bounding Mechanics: 1) Pre-Flight Token Counting: Compute exact token counts locally before network dispatch using high-performance tokenizers (e.g. `JTokkit` for Java). 2) Asymmetric Reservation Protocol: The completion budget must remain strictly reserved; choking completion tokens causes mid-sentence cutoff (`finish_reason: length`), corrupting JSON payloads. 3) Dynamic Context Degradation: If a long query breaches budget, shed the lowest-ranked RAG chunk first, then truncate conversation history, never truncating the core system instructions.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Sıkı Token Bütçe Yöneticisi (Token Budget Enforcer)
public class TokenBudgetEnforcer {
    private final int totalBudget;
    private final int reservedForCompletion;

    public TokenBudgetEnforcer(int totalBudget, int reservedForCompletion) {
        this.totalBudget = totalBudget;
        this.reservedForCompletion = reservedForCompletion;
    }

    public int getAvailablePromptTokens() {
        return totalBudget - reservedForCompletion;
    }

    public boolean validateBudget(int currentPromptTokens) {
        return currentPromptTokens <= getAvailablePromptTokens();
    }
}

void main() {
    var enforcer = new TokenBudgetEnforcer(4096, 1024); // 4096 toplam, 1024 yanıta ayrıldı
    int promptTokens = 2500;

    IO.println("İzin Verilen Maksimum Prompt Token: " + enforcer.getAvailablePromptTokens());
    IO.println("Bütçe Kontrolü (2500 token): " + (enforcer.validateBudget(promptTokens) ? "✅
GEÇERLİ" : "❌ BÜTÇE AŞILDI"));
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

JSON Bozulma Tuzağı: Modelden JSON beklerken `max_tokens` değerini çok küçük tutarsanız (ör. 100 token), model JSON'ı kapatamadan kesilir (`{"name": "Ahmet", "stat...`) ve Jackson parser JSONSyntaxException patlatır! JSON çıktılarında completion bütçesi daima cömert tutulmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

JSON truncation disaster: Constraining `max_tokens` too tightly when expecting structured outputs causes the generation to abort mid-object (`{"status": "SUCC...`), yielding malformed JSON that crashes downstream deserializers. Always provision ample completion headroom.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Token bütçeleme, beklenmeyen fatura patlamalarını ve bağlam aşımını önler.
- Yanıta ayrılan completion token bütçesi JSON bozulmalarını önlemek için daima korunmalıdır.
- Token sınırına yaklaşıldığında sistem promptu değil, en eski RAG parçaları budanmalıdır.

🎯 Key Takeaways for the Interview

- Token budgeting prevents unexpected billing spikes and context window overflows.
- Completion token reservations must be safeguarded to avoid truncated JSON payloads.
- When shedding tokens, prune low-ranked RAG chunks first, never core system instructions.